

# 5. GIT BRANCHES: A BOOKMARK, NOT A COPY

Try an idea without touching what works

## ANALOGY: THE FAMILY RECIPE BOOK



Mila

① I want to try the soup with chili.

③ I'll use my own bookmark and new pages.

② Not in the book we all cook from!

④ Fine. If it's good, we keep the pages.



Grandpa

Nobody copies the book. They agree on this:

- each new page says which page came before it
- a bookmark only marks the page you are on
- good soup: the new pages join the family book
- bad soup: throw the bookmark away

A branch is the bookmark: a name that points at one commit. Creating it copies no files.

## THE SAME STORY IN GIT

|                 | RECIPE BOOK                       | GIT                                                               |
|-----------------|-----------------------------------|-------------------------------------------------------------------|
| PAGE            | one written page of the book      | <b>commit</b> : a snapshot of all files + a link to its parent(s) |
| BOOKMARK        | a small strip that marks a page   | <b>branch</b> : a movable name pointing at one commit             |
| FAMILY BOOKMARK | the one everyone cooks from       | <b>main</b> , the usual name of the default branch                |
| YOUR PLACE      | the bookmark you are working from | <b>HEAD</b> : points at the branch you are on                     |

## LOOK FOR YOURSELF

```
$ cat .git/HEAD
ref: refs/heads/main
HEAD just names the branch you are on.

$ cat .git/refs/heads/main
d7b9e90f1dac36702a0ff2e6c705db70c0ce6259
The whole branch: one commit ID. (Git may also pack refs
into one file.)
```

## ONE EXPERIMENT, START TO FINISH

### 1. NEW BOOKMARK

`git switch -c feature`  
new branch here, HEAD moves to it



### 2. WRITE PAGES

`git add .`  
`git commit -m "chili"`  
the bookmark moves on



### 3. BACK TO FAMILY

`git switch main`  
files return to main's snapshot



### 4. JOIN THE PAGES

`git merge feature`  
main now has the chili

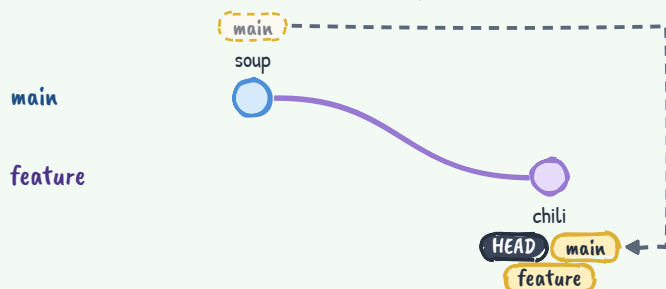


### 5. DROP THE BOOKMARK

`git branch -d feature`  
refuses if unmerged; `-D` forces

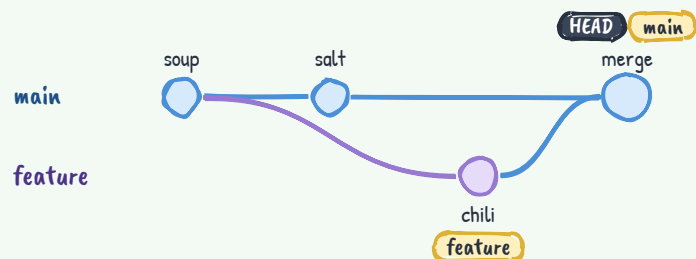


## MERGE, CASE 1: FAST-FORWARD



Main had nothing new: **main** just slides to chili. No new commit.

## MERGE, CASE 2: A MERGE COMMIT



Both moved on: a new commit with **two parents**. See it: `git log --graph`



**CONFLICT:** Both sides changed the same lines. Git marks them with `<<<<<<<` and waits for a human to choose.



## REMEMBER:

A branch is a bookmark on a commit, not a copy of your files. Make them freely, merge the good ones.